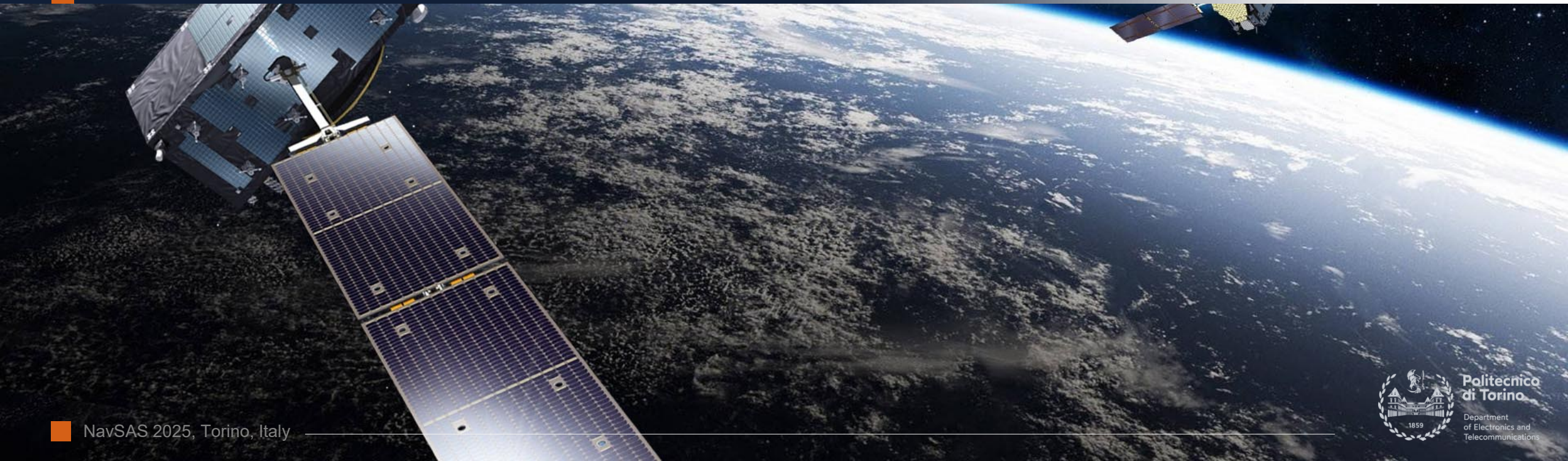
Lab Session 2: State Estimation

Laboratory on Least Squares position and clock bias estimation | **LAB BRIEFING**

Andrea **Nardin,** Simone **Zocca,** prof. Alex **Minetto**
name.surname@polito.it



Politecnico di Torino
Department of Electronics and Telecommunications

Estimating Position from GNSS observables

GNSS radionavigation is based on a set of measurements, also referred to as **observables,** that a receiver obtains by receiving and processing satellites radionavigation **signals**.

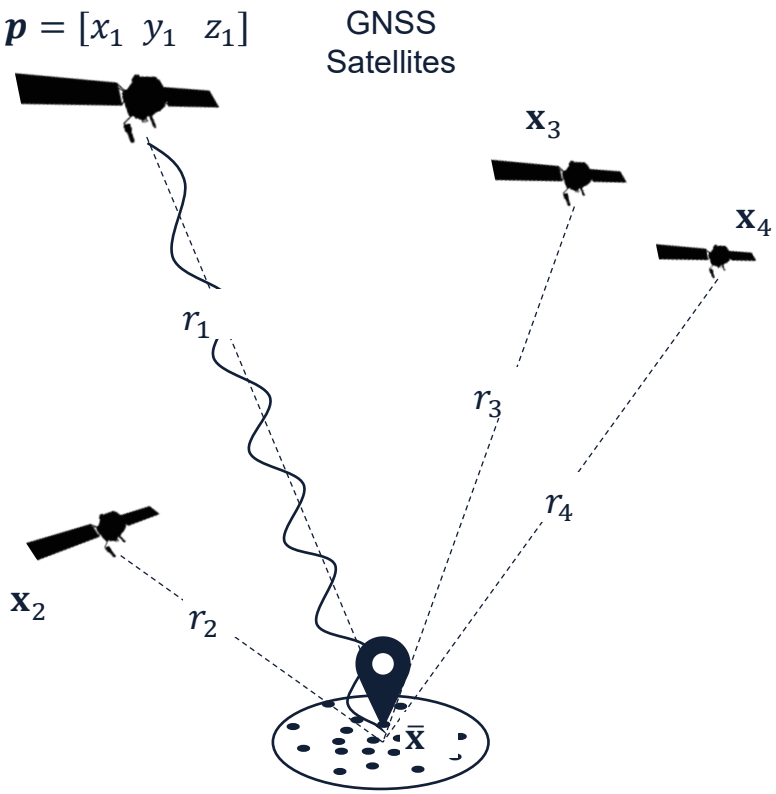
Observables include **pseudorange** and **pseudorange-rate / Doppler shift** measurements for each "visible" satellite (as in Android raw measurements, see Lab Session 1)

Observables depend on the **state** of the receiver. By inverting this known observable-state relationships, through a sufficient number of **observables** we can estimate the **receiver state,** also referred to **position, velocity and time (PVT)** solution.

Observable-state relationship example (pseudoranges):

$$\rho_1 = \overbrace{\sqrt{(x_1 - x)^2 + (y_1 - y)^2 + (z_1 - z)^2}}^{r_1} - b_u$$

Measured — **Known A-priori** — **Uknown**



Snapshot at a time instant n of the radionavigation problem for a GNSS receiver and a set of GNSS satellites visible to the receiver.

Estimating Position from GNSS observables

GNSS radionavigation is based on a set of measurements, also referred to as **observables,** that a receiver obtains by receiving and processing satellites radionavigation **signals**.

Observables include **pseudorange** and **pseudorange-rate / Doppler shift** measurements for each "visible" satellite (as in Android raw measurements, see Lab Session 1)

Observables depend on the **state** of the receiver. By inverting this known observable-state relationships, through a sufficient number of **observables** we can estimate the **receiver state,** also referred to **position, velocity and time (PVT)** solution.

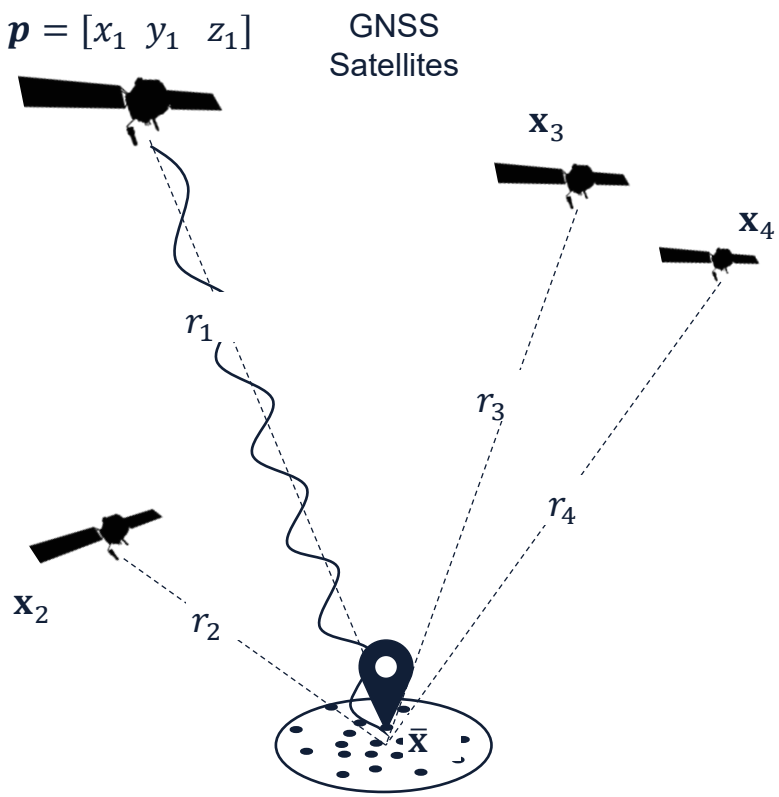
Observable-state relationship example (pseudoranges):

$$\mathbf{x} = [x\ y\ z\ b_u] = [\boldsymbol{p}\ b_u]$$

$$\rho_1 = |\boldsymbol{p_1} - \boldsymbol{p}| - b_u$$

Measured — **Known A-priori** — **Uknown**

How to estimate the unknowns from observables ?



Snapshot at a time instant n of the radionavigation problem for a GNSS receiver and a set of GNSS satellites visible to the receiver.

Estimating Position from GNSS observables

We need a criterion based on which we estimate a solution. A common choice is that of **Least Squares (LS)** method**.**

- LS minimizes the squared **residuals**, which are the difference between observed measurements $\boldsymbol{\rho}$ and nominal measurements $\widehat{\boldsymbol{\rho}}$:

$$\Delta\boldsymbol{\rho} = \boldsymbol{\rho} - \widehat{\boldsymbol{\rho}}$$

- The norm of residuals gives an idea of how far away we are from the minimum.
- Then, we need the «direction» in which we should adjust the solution in order to find the minimum. This «direction» is given by the derivative of residuals
- Since the equations are non linear, we need to linearize the system so we can solve it in matrix form
- So, we start from an arbitrary **linearization point** and then we iterate towards the minimum

Least Squares applied to Single Point Positioning (SPP)

- **Position, Velocity and Time** (PVT) solution can be estimated by linearizing a multilateration problem through the method of **Least Squares** (LS):

$$\Delta \mathbf{x} = \left(\mathbf{H}^{\mathrm{T}}\mathbf{H}\right)^{-1}\mathbf{H}^{\mathrm{T}}\Delta\boldsymbol{\rho}$$

- One possible way to solve for the PVT is to **iteratively** apply the LS correction by updating the **linearization point** at each iteration to obtain a more accurate solution
- At each iteration, the estimated PVT is expected to be closer to the actual user location

Least Squares applied to Single Point Positioning (SPP)

- **Position, Velocity and Time** (PVT) solution can be estimated by linearizing a multilateration problem through the method of **Least Squares** (LS):

$$\Delta \mathbf{x}^k = \left((\mathbf{H}^k)^{\mathrm{T}} \mathbf{H}^k \right)^{-1} (\mathbf{H}^k)^{\mathrm{T}} \Delta \boldsymbol{\rho}^k$$

- One possible way to solve for the PVT is to iteratively apply the LMS algorithm by updating the **linearization point** at each iteration *k* to obtain a more accurate solution
- At each iteration, the estimated PVT is expected to be closer to the actual user location

Least Squares applied to Single Point Positioning (SPP)

- **Position, Velocity and Time** (PVT) solution can be estimated by linearizing a multilateration problem through the method of **Least Squares** (LS):

$$\Delta \mathbf{x}_n^k = \left((\mathbf{H}_n^k)^{\mathrm{T}} \mathbf{H}_n^k \right)^{-1} (\mathbf{H}_n^k)^{\mathrm{T}} \Delta \boldsymbol{\rho}_n^k$$

- One possible way to solve for the PVT is to iteratively apply the LMS algorithm by updating the **linearization point** at each iteration *k* to obtain a more accurate solution
- At each iteration, the estimated PVT is expected to be closer to the actual user location
- At each epoch *n*, new measurements are processed and the iterative LS restarts
- In this lab we will process pseudorange to solve for position and clock bias only.

Iterative Least Squares

- For each epoch n, a number of iterations $k = 1, \dots, K$ is performed. K has to be chosen in order to obtain a "stable" solution (generally $K = 8$ is sufficient)
- The **initial** linearization point, $\hat{\mathbf{x}}_n^0$, could be the same at each time n (but why should be?)

Initialization	$\hat{\mathbf{x}}_n^0$
k-th iteration At n-th epoch	$\Delta\boldsymbol{\rho}_n^k = \boldsymbol{\rho}_n - \hat{\boldsymbol{\rho}}_n^k$
	$\Delta\mathbf{x}_n^k = \left((\mathbf{H}_n^k)^{\mathrm{T}}\mathbf{H}_n^k\right)^{-1}(\mathbf{H}_n^k)^{\mathrm{T}}\Delta\boldsymbol{\rho}_n^k$
	$\hat{\mathbf{x}}_n^{k+1} = \hat{\mathbf{x}}_n^k - \Delta\mathbf{x}_n^k$

- $\boldsymbol{\rho}_n$: Measured/observed pseudoranges
- $\hat{\boldsymbol{\rho}}_n^k$: Nominal pseudoranges from $\hat{\boldsymbol{x}}_n^k$ to satellite position:

$$\hat{\rho}_{j,n} = \left|\boldsymbol{p}_{j,n} - \hat{\boldsymbol{p}}_n^k\right| - b_{u,n}^k - c \cdot \delta t_{j,n}$$

Satellite clock bias

- Pseudoranges $\boldsymbol{\rho}$ and satellite positions $\boldsymbol{p}_j$ are updated at **each epoch $\boldsymbol{n}$**
- Nominal measurements $\hat{\boldsymbol{\rho}}$, $\mathbf{H}$ matrix and linearization point $\hat{\mathbf{x}}$ are updated at **each iteration $\boldsymbol{k}$ and each epoch $\boldsymbol{n}$**

Iterative Least Squares

- The **number of visible satellites** J, is **not constant** over the observation timespan
- At each iteration k, $\mathbf{H}_n^k$ is the geometrical matrix obtained from the previous estimated position:

$$\mathbf{H}_n^k = \begin{bmatrix} a_{x,1_n}^k & a_{y,1_n}^k & a_{z,1_n}^k & 1 \\ a_{x,2_n}^k & a_{y,2_n}^k & a_{z,2_n}^k & 1 \\ a_{x,3_n}^k & a_{y,3_n}^k & a_{z,3_n}^k & 1 \\ \vdots & \vdots & \vdots & \vdots \\ a_{x,J_n}^k & a_{y,J_n}^k & a_{z,J_n}^k & 1 \end{bmatrix}$$

Each row corresponds to one satellite

Satellite coordinates are kept fixed for any k

$$a_{x,j,n}^k = \left[\frac{x_{j,n} - \hat{x}_n^k}{r_{j,n}}\right], \quad a_{y,j_n} = \cdots, \quad a_{z,j_n} = \cdots$$

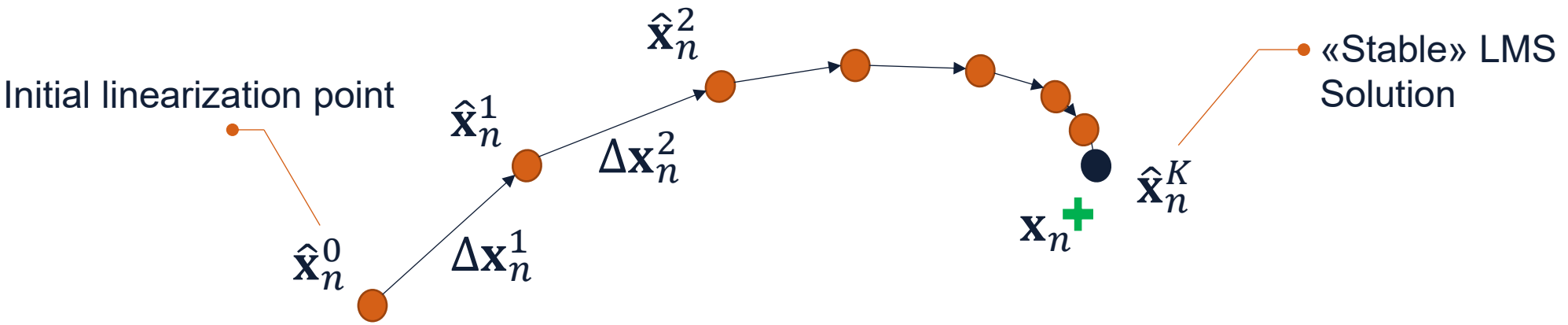
$$r_{j,n} = \left|\boldsymbol{p}_{j,n} - \hat{\boldsymbol{p}}_n^k\right|$$

Geometrical range from $\hat{\boldsymbol{x}}_n^k$ and satellite position ($\neq \hat{\rho}_{j,n}$)

Iterative Least Squares

- By adding $\Delta\mathbf{x}_n^k$ to the estimated position at the previous iteration, the position estimate is updated, and a new solution is found.

$$\hat{\mathbf{x}}_n^k = \hat{\mathbf{x}}_n^{k-1} - \Delta\mathbf{x}_n^k$$

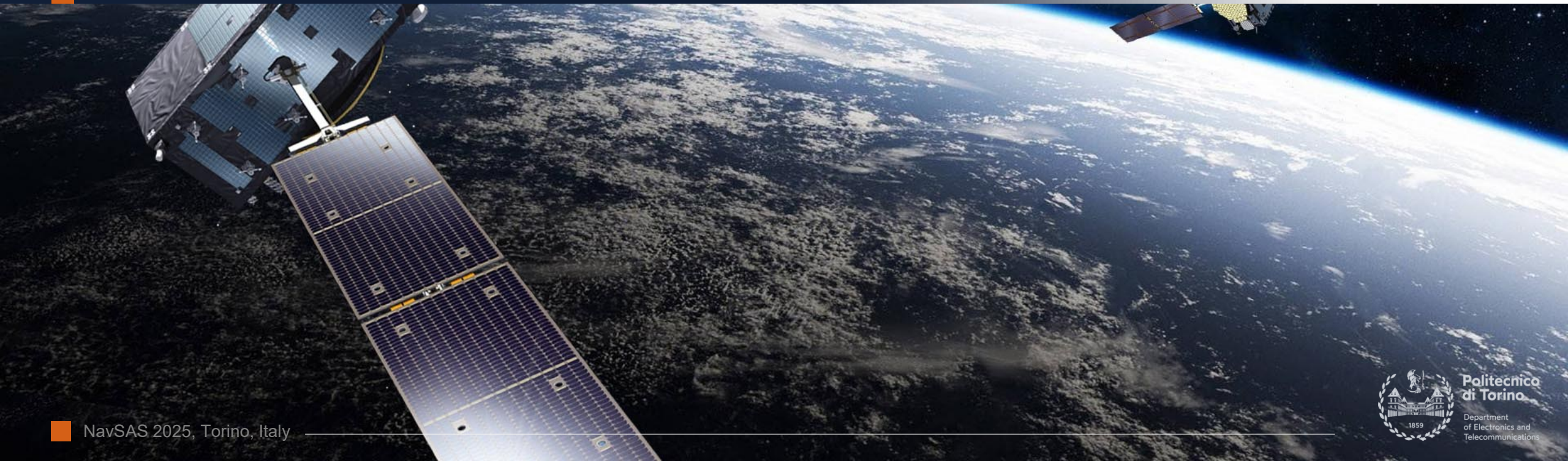


CONVERGENCE OF THE SOLUTION

Lab Session 2: State Estimation

Laboratory on Least Squares position and clock bias estimation | **LAB BRIEFING**

Andrea **Nardin,** Simone **Zocca,** prof. Alex **Minetto**
name.surname@polito.it



Politecnico di Torino
Department of Electronics and Telecommunications

A. State estimation

TASK A.1

Find the script «pvtCore» in the MATLAB code from Lab 1 and get accustomed to the different variables and the calling functions

TASK A.2

Implement the core block of PVT estimation using Least Squares algorithm

Hint: *You'll see that the iterative structure of the code and some of the variables you need have already been prepared for you, explore the script and try to compute the other quantities you need*

TASK A.3

To test your code, uncomment the line calling the «GpsWlsPvt» function (and comment the other line) from the **main** script and run it using one of the demo datasets

```
function [xHat,dRho,H,dx] = pvtCore(pseudoranges,svPos,svClockBias,xHat,Wpr)
%
% Inputs:
%       pseudoranges:   pseudorange vector, one for each sat
%       svPos:          3D satelite positions
%       svClockBias:    satellite clock bias
%       xHat:           linearization point
%       Wpr:            matrix of weights
%
% Outputs:
%       xHat:           updated solution (updated lin. point)
%       dRho:           residuals
%       H:              H matrix
%       dx:             delta_x, i.e. update of lin. point
```

```
%% compute WLS position and velocity
%gpsPvt = GpsWlsPvt(gnssMeas,allGpsEph);
gpsPvt = [];
```

B. Lab session | Custom Datasets

TASK B.1

First, verify the performance of LS on one of the **static** datasets you collected

TASK B.2

Then, apply one of the data filters and repeat the previous point.
What is the difference? Did the performance improve or not? Why?

For your final report, make sure to comment on the plot you generated, highlighting specific characteristics of the processed dataset. Use at least two datasets obtained under different visibility conditions to build a meaningful comparison.